Mrunal Samrutwar

UsnNo: CS23082

Aim : To implement the KMeans clustering algorithm and analyze the clustering result using appropritae evaluation techniques

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
path="/content/drive/MyDrive/ML Dataset/Mall_Customers.csv"
df=pd.read_csv(path)
df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Next steps: Generate code with df New interactive sheet

```
X=df.iloc[:,[3,4]].values
print(X)
```

```
[ 78  22]
[ 78  90]
[ 78  17]
[ 78  88]
[ 78  20]
[ 78  76]
[ 78  16]
[ 78  89]
[ 78   1]
[ 78  78]
[ 78   1]
[ 78  73]
[ 79  35]
[ 79  83]
[ 81   5]
[ 81  93]
[ 85  26]
[ 85  75]
[ 86  20]
[ 86  95]
[ 87  27]
[ 87  63]
[ 87  13]
[ 87  75]
```

print(X_scaled)

```
[ 0.16947227 -0.08540743]
[ 0.16947227 -0.00776431]
[ 0.16947227 -0.27951524]
[ 0.16947227  0.34162973]
[ 0.24581112 -0.27951524]
[ 0.24581112  0.26398661]
[ 0.24581112  0.22516505]
[ 0.24581112 -0.39597992]
[ 0.32214998  0.30280817]
[ 0.32214998  1.58391968]
[ 0.36031941 -0.82301709]
[ 0.36031941  1.04041783]
[ 0.39848884 -0.59008772]
[ 0.39848884  1.73920592]
[ 0.39848884 -1.52180518]
[ 0.39848884  0.96277471]
[ 0.39848884 -1.5994483 ]
[ 0.39848884  0.96277471]
[ 0.43665827 -0.62890928]
[ 0.43665827  0.80748846]
[ 0.4748277  -1.75473454]
[ 0.4748277   1.46745499]
[ 0.4748277  -1.67709142]
[ 0.4748277   0.88513158]
[ 0.51299713 -1.56062674]
[ 0.51299713  0.84631002]
[ 0.55116656 -1.75473454]
[ 0.55116656  1.6615628 ]
[ 0.58933599 -0.39597992]
[ 0.58933599  1.42863343]
[ 0.62750542 -1.48298362]
[ 0.62750542  1.81684904]
[ 0.62750542 -0.55126616]
[ 0.62750542  0.92395314]
[ 0.66567484 -1.09476801]
[ 0.66567484  1.54509812]
[ 0.66567484 -1.28887582]
[ 0.66567484  1.46745499]
[ 0.66567484 -1.17241113]
[ 0.66567484  1.00159627]
[ 0.66567484 -1.32769738]
[ 0.66567484  1.50627656]
[ 0.66567484 -1.91002079]
[ 0.66567484  1.07923939]
[ 0.66567484 -1.91002079]
[ 0.66567484  0.88513158]
[ 0.70384427 -0.59008772]
[ 0.70384427  1.27334719]
[ 0.78018313 -1.75473454]
[ 0.78018313  1.6615628 ]
[ 0.93286085 -0.93948177]
[ 0.93286085  0.96277471]
[ 0.97103028 -1.17241113]
[ 0.97103028  1.73920592]
[ 1.00919971 -0.90066021]
[ 1.00919971  0.49691598]
[ 1.00919971 -1.44416206]
[ 1.00919971  0.96277471]
```

```
scaler=StandardScaler()
X_scaled=scaler.fit_transform(X)
```

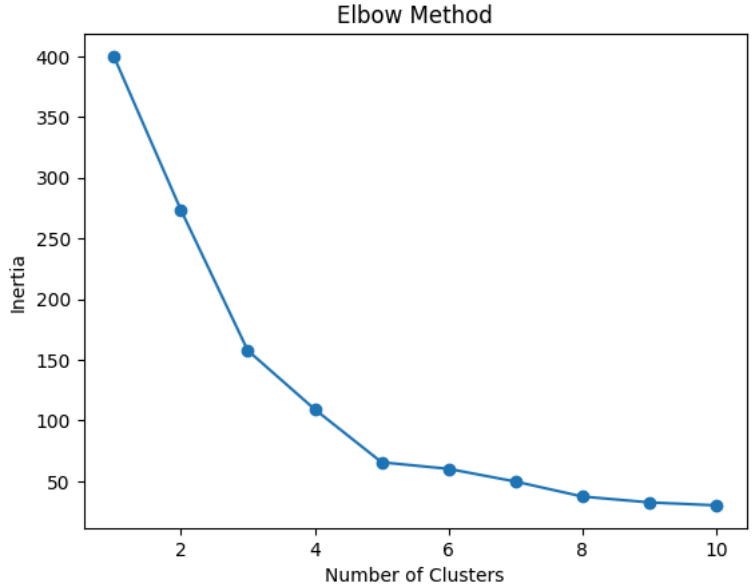
```
inertia_values = []
for k in range(1,11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    inertia_values.append(kmeans.inertia_)

    print(inertia_values)
    plt.figure()
```

```
[399.99999999999994]
[399.99999999999994, 273.66888662642003]
[399.99999999999994, 273.66888662642003, 157.70400815035939]
[399.99999999999994, 273.66888662642003, 157.70400815035939, 109.22822707921345]
[399.99999999999994, 273.66888662642003, 157.70400815035939, 109.22822707921345, 65.56840815571681]
[399.99999999999994, 273.66888662642003, 157.70400815035939, 109.22822707921345, 65.56840815571681, 60.132874871934206]
[399.99999999999994, 273.66888662642003, 157.70400815035939, 109.22822707921345, 65.56840815571681, 60.132874871934206, 49.668244837367965]
[399.99999999999994, 273.66888662642003, 157.70400815035939, 109.22822707921345, 65.56840815571681, 60.132874871934206, 49.668244837367965, 37.
[399.99999999999994, 273.66888662642003, 157.70400815035939, 109.22822707921345, 65.56840815571681, 60.132874871934206, 49.668244837367965, 37.
[399.99999999999994, 273.66888662642003, 157.70400815035939, 109.22822707921345, 65.56840815571681, 60.132874871934206, 49.668244837367965, 37.
<Figure size 640x480 with 0 Axes>
<Figure size 640x480 with 0 Axes>
<Figure size 640x480 with 0 Axes>
<Figure size 640x480 with 0 Axes>
<Figure size 640x480 with 0 Axes>
<Figure size 640x480 with 0 Axes>
<Figure size 640x480 with 0 Axes>
<Figure size 640x480 with 0 Axes>
<Figure size 640x480 with 0 Axes>
<Figure size 640x480 with 0 Axes>
```

```
print(inertia_values)
plt.figure()
plt.plot(range(1,11),inertia_values,marker='o')
plt.xlabel('Number of Clusters')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.show()
```

```
[399.99999999999994, 273.66888662642003, 157.70400815035939, 109.22822707921345, 65.56840815571681, 60.132874871934206, 49.668244837367965, 37.
```




```
kmeans = KMeans(n_clusters=5, random_state=42)
cluster=kmeans.fit_predict(X_scaled)
df['cluster']=cluster
print(df.head())
```

```
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)  \
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

   cluster
0        4
1        2
2        4
3        2
4        4
```

```
df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)	cluster
0	1	Male	19	15	39	4
1	2	Male	21	15	81	2
2	3	Female	20	16	6	4
3	4	Female	23	16	77	2
4	5	Female	31	17	40	4

Next steps: Generate code with df New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 6 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
 5   cluster                 200 non-null    int32
dtypes: int32(1), int64(4), object(1)
memory usage: 8.7+ KB
```

```
df.shape
```

```
(200, 6)
```

```
print("Inertia:",kmeans.inertia_)
sil_score=silhouette_score(X_scaled,cluster)
print("Silhouette Score:",sil_score)
```

```
Inertia: 65.56840815571681
Silhouette Score: 0.5546571631111091
```

```
plt.figure()
sns.scatterplot(x=df['Annual Income (k$)'],y='Spending Score (1-100)', hue='cluster', data=df, palette='Set1')
centroids=scaler.inverse_transform(kmeans.cluster_centers_)
plt.scatter(centroids[:,0], centroids[:,1], marker='X', color='red', s=200, label='centroids')
plt.title('Customer segmentation using KMeans')
plt.legend()
plt.show()
```

```
kmeans = KMeans(n_clusters=4, random_state=42)
cluster=kmeans.fit_predict(X_scaled)
df['cluster']=cluster
print(df.head())
```

```
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)  \
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

   cluster
0        0
1        2
2        0
3        2
4        0
```

```
print("Inertia:",kmeans.inertia_)
sil_score=silhouette_score(X_scaled,cluster)
print("Silhouette Score:",sil_score)
```

```
Inertia: 157.70400815035939
Silhouette Score: 0.46658474419000145
```

```
kmeans = KMeans(n_clusters=7, random_state=42)
cluster=kmeans.fit_predict(X_scaled)
df['cluster']=cluster
print(df.head())
```

```
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)  \
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

   cluster
0        5
1        2
2        4
3        2
4        5
```

```
print("Inertia:",kmeans.inertia_)
sil_score=silhouette_score(X_scaled,cluster)
print("Silhouette Score:",sil_score)
```

```
Inertia: 49.668244837367965
Silhouette Score: 0.50200146805547
```

```
kmeans = KMeans(n_clusters=8, random_state=42)
cluster=kmeans.fit_predict(X_scaled)
df['cluster']=cluster
print(df.head())
```

```
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)  \
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

   cluster
0        4
1        3
2        4
3        3
4        4
```

```
print("Inertia:",kmeans.inertia_)
sil_score=silhouette_score(X_scaled,cluster)
print("Silhouette Score:",sil_score)
```

```
Inertia: 37.31912287833882
Silhouette Score: 0.4550112502601921
```